MEDICAL INVENTORY MANAGEMENT

Submitted by

PANCHAGNULA MRIDAH SHIVA KUMAR

(RA2411003020287)

PRASHANTH L (RA2411003020299)

JOSH THOMAS (RA2411003020305)

Under the guidance of

Mrs. C. GEETHA

(Assistant Professor, Department of Computer Science and Engineering)

21CSC205P/DATA BASE MANAGEMENT SYSTEM PROJECT REPORT

IV SEMESTER/II YEAR

COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING AND TECHNOLOGY

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY RAMAPURAM, CHENNAI

APRIL 2026

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY

(Deemed to be University U/S 3 of UGC Act, 1956)

BONAFIDE CERTIFICATE

Certified that this project report titled **"MEDICAL INVENTORY MANAGEMENT"** is the bonafide work of **PANCHAGNULA MRIDAH SHIVA KUMAR (RA2411003020297), PRASHANTH L (RA2411003020299), JOSH THOMAS (RA2411003020305),** who carried out the project work under my supervision. Certified further, that to the best of my knowledge the work reported herein does not form any other project report or dissertation on the basis of which a degree or award was conferred on an occasion on this or any other candidate. This project work confirms **21CSC205P/DATA BASE MANAGEMENT SYSTEM**, IV Semester, II year, 2024.

SIGNATURE

Mrs. C. GEETHA, M.E., Ph.D.

Assistant Professor

Computer Science and Engineering,
SRM Institute of Science and Technology,

Ramapuram, Chennai.

SIGNATURE

Dr. K. RAJA, M.E., Ph.D.

Professor and Head

Computer Science and Engineering, SRM Institute of Science and Technology,

Ramapuram, Chennai.

Submitted for the project viva-voce held on _____at SRM Institute of Science and Technology, Ramapuram, Chennai.

INTERNAL EXAMINER 1

INTERNAL EXAMINER 2

SRM INSTITUTE OF SCIENCE AND TECHNOLOGY
RAMAPURAM, CHENNAI

DECLARATION

We hereby declare that the entire work contained in this project report titled **"PAYROLL MANAGEMENT SYSTEM"** has been carried out by **PANCHAGNULA MRIDAH SHIVA KUMAR (RA2411003020287), PRASHANTH L (RA2411003020299), JOSH THOMAS (RA2411003020305),** at SRM Institute of Science and Technology, Ramapuram, Chennai, under the guidance of **Mrs. C. GEETHA ASSISTANT PROFESSOR**, Department of Computer Science and Engineering.

Place: Chennai **MRIDAH SHIVA KUMAR**

Date: **PRASHANTH L**

JOSH THOMAS

ACKNOWLEDGEMENT

We place on record our deep sense of gratitude to our lionized Chairman **Dr. R. SHIVAKUMAR, MBBS, MD**, for providing us with the requisite infrastructure throughout the course.

We take this opportunity to extend our hearty and sincere thanks to our Dean, **Dr. M. SAKTHI GANESH** for guiding us in accomplishing this project.

We thank our honorable Head of the Department **Dr. K. RAJA** for his constant motivation and unwavering support.

We express our hearty and sincere thanks to our guide **Mrs. C. GEETHA, Assistant Professor, CSE** for his encouragement, consecutive criticism and constant guidance throughout this project work.

Our thanks to the Teaching and non-Teaching staff of the Department of Computer Science and Engineering of SRM Institute of Science and Technology, Chennai, for providing necessary resources for our project.

ABSTRACT

The Medical Inventory Management System is a web-based application developed to efficiently manage and monitor the inventory of medicines in healthcare institutions such as hospitals and pharmacies. In traditional systems, inventory management is often performed manually using registers or basic spreadsheets, which are prone to human errors, data redundancy, and inefficiencies in tracking stock levels and expiry dates. To overcome these limitations, the proposed system provides an automated and centralized solution that ensures accurate and real-time management of medical inventory. The application is implemented using Flask as the backend framework, with Python for application logic and SQLite as the database for storing structured data. The system maintains detailed records of medicines, including batch numbers, suppliers, stock quantities, and expiry dates, and supports essential operations such as creating, updating, deleting, and retrieving data through a user-friendly web interface. It also incorporates features such as automatic stock updates after sales and purchases, alerts for low stock levels, and identification of expired or near-expiry medicines to ensure better inventory control and patient safety. The database design follows normalization principles to reduce redundancy and maintain data integrity, using primary and foreign key constraints to establish relationships between entities. Furthermore, the system adopts a modular architecture and uses RESTful APIs to enable efficient communication between the frontend and backend components. Overall, this project demonstrates the practical implementation of database management concepts in solving real-world healthcare challenges, improving operational efficiency, and providing a scalable and reliable solution for medical inventory management.

TABLE OF CONTENTS

Chapter no.	Title	Page No.
	Abstract	v
	Table of Contents	vi
	List of Figures	vii
	List of Abbreviations	viii
1	Introduction 1.1 Introduction 1.2 Problem Statement 1.3 Objective 1.4 Scope And Motivation	1
2	Existing System	4
3	Design 3.1 E-R Design 3.2 Front End Design	6
4	Proposed Methodology 4.1 Module Description 4.2 Database Connectivity	13
5	Implementation 5.1 Back End Via SQL 5.2 Front End Via Python	17
6	Result And Discussion 6.1 System Functionality Evaluation 6.2 Performance Evaluation 6.3 User Feedback And Adoption 6.4 Future Enhancements	29
7	Conclusion	39
8	References	40

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	ER DIAGRAM OF PMS	8
3.2	USE CASE DIAGRAM OF PMS	9
3.3	ARCHITECTURE DIAGRAM OF PMS	10
5.1	DASHBOARD PAGE	24
5.2	ORGANISATION PAGE	24
5.3	DEPARTMENT PAGE	25
5.4	EMPLOYEE PERSONAL INFORMATION DETAILS	25
5.5	EARNINGS DEDUCTION TABLE	26
5.6	ADMIN TABLE	26
5.7	DEPARTMENT TABLE	27
5.8	ORAGNISATION TABLE	27

LIST OF ABBREVIATIONS

API: Application Programming Interface

UI : User Interface

RDBMS : Relational Database Management System

DBMS : Database Management System

SQL : Structured Query Language

E-R : Entity Relationship

CRUD : Create, Read, Update, Delete

HTML: Hyper Text Markup Language

CHAPTER 1

INTRODUCTION

1.1 Introduction:

In modern healthcare systems, efficient management of medical inventory is essential to ensure the availability of medicines and proper patient care. Traditional inventory systems rely heavily on manual record-keeping, which is prone to errors and inefficiencies.

The Medical Inventory Management System is designed to automate the process of inventory management using a web-based platform. The system enables users to maintain accurate records of medicines, suppliers, and stock levels, thereby improving efficiency and reducing manual effort.

1.2 Problem statement:

The management of medical inventory in hospitals and pharmacies is a critical task that directly impacts patient care and operational efficiency. In many healthcare institutions, inventory is still managed using manual methods such as paper-based records or basic spreadsheet systems. These traditional approaches are highly prone to human errors, data redundancy, and inconsistencies, making it difficult to maintain accurate and up-to-date information.

One of the major challenges in the existing system is the inability to track stock levels in real time, which often leads to situations such as stock shortages or overstocking of medicines. Additionally, the absence of proper monitoring mechanisms for batch numbers and expiry dates increases the risk of expired medicines being used, which can have serious consequences on patient safety. Managing supplier information and purchase history is also cumbersome, making it difficult to analyse procurement patterns and maintain accountability.

Furthermore, the lack of a centralized system results in scattered data, slow retrieval of information, and inefficient reporting processes. These limitations highlight the need for a reliable, automated, and scalable system that can streamline inventory operations, ensure data accuracy, and provide real-time insights for effective decision-making.

1.3 Objective:

The primary objective of this project is to design and develop a web-based Medical Inventory Management System that provides an efficient and reliable solution for managing medical stock. The system aims to automate the process of inventory management and eliminate the drawbacks associated with manual systems.

The specific objectives of the project are:

- To develop a centralized database system for storing and managing medical inventory data
- To maintain accurate records of medicines, including details such as category, price, batch number, and expiry date
- To manage supplier information and track purchase transactions effectively
- To implement real-time stock monitoring and automatic updates after sales and purchases
- To provide alerts for low stock levels and expired or near-expiry medicines
- To ensure data integrity and consistency using primary and foreign key constraints
- To implement CRUD operations through a user-friendly web interface
- To improve operational efficiency and reduce manual errors in inventory handling
- To demonstrate the practical application of DBMS concepts in a real-world scenario

1.4 Scope and Motivation:

Scope

The scope of the Medical Inventory Management System is focused on providing an efficient solution for managing medical inventory in small to medium-scale healthcare environments such as clinics, pharmacies, and hospitals. The system covers key functionalities including medicine management, supplier management, inventory tracking, and sales and purchase recording.

The application allows users to maintain detailed records of medicines, monitor stock levels, track expiry dates, and generate alerts for critical situations such as low stock or expired items. It also supports data retrieval and reporting, enabling users to analyse inventory status and make informed decisions.

However, the system is limited to inventory management and does not include advanced features such as patient record management, billing systems, or integration with external healthcare platforms. These features can be considered for future enhancements.

Motivation

The motivation behind developing this system arises from the need to overcome the inefficiencies and risks associated with manual inventory management in the healthcare sector. Accurate availability of medicines is crucial for ensuring proper patient care,errors in inventory management can lead to serious consequences.

By automating the inventory process, the system reduces human effort, minimizes errors, and improves overall efficiency. The use of modern technologies such as Flask and SQLite provides a scalable and easy-to-use solution that can be adapted to different healthcare environments.

Additionally, this project serves as a practical implementation of database management concepts learned in academics, allowing the application of theoretical knowledge to solve real-world problems. It also enhances technical skills in web development, database design, and system integration.

CHAPTER 2

EXISTING SYSTEM

In many healthcare institutions such as hospitals, clinics, and pharmacies, the management of medical inventory is still carried out using traditional methods such as manual record-keeping or basic spreadsheet applications. These methods involve maintaining physical registers or simple digital files to record details related to medicines, suppliers, stock levels, and transactions. While these approaches may be sufficient for very small-scale operations, they become highly inefficient and unreliable as the volume of data increases.

In a typical manual system, information about medicines such as name, quantity, batch number, and expiry date is recorded by staff members in registers or spreadsheets. Similarly, purchase and sales transactions are documented separately, often without proper integration. This leads to fragmented data storage, where information is scattered across multiple sources, making it difficult to maintain consistency and accuracy.

One of the major drawbacks of the existing system is the high dependency on human effort. Since all entries are performed manually, there is a significant risk of human errors such as incorrect data entry, duplication of records, and omission of important details. These errors can accumulate over time and lead to serious discrepancies in inventory records.

Another critical limitation is the lack of real-time stock monitoring. In manual systems, stock levels are updated only when records are reviewed or modified, which may not happen frequently. As a result, there is no accurate and up-to-date view of the available inventory at any given time. This can lead to situations where essential medicines run out of stock unexpectedly, affecting patient care, or excess stock is maintained unnecessarily, leading to wastage of resources.

The management of expiry dates is also a significant challenge in the existing system. Medicines often have specific shelf lives, and it is essential to ensure that expired or near-expiry products are not used or sold. However, in manual systems, tracking expiry dates requires constant monitoring and manual verification, which is both time-consuming and error-prone. This increases the risk of expired medicines being dispensed, which can have serious health implications.

Supplier management is another area where the existing system falls short. Information related to

suppliers, such as contact details, supply history, and transaction records, is often maintained separately or inconsistently. This makes it difficult to track supplier performance, manage procurement efficiently, and maintain proper communication. Additionally, the absence of integrated

purchase records makes it challenging to analyse buying patterns and optimize inventory levels.

Data retrieval and reporting are also inefficient in manual systems. Generating reports such as stock summaries, sales analysis, or purchase history requires significant time and effort, as data must be collected and compiled manually from different sources. This not only delays decision-making but also increases the chances of inaccuracies in reports.

Furthermore, the lack of a centralized system results in poor coordination between different departments. Since data is not stored in a unified database, multiple users may maintain separate records, leading to inconsistencies and duplication. This makes it difficult to ensure data integrity and maintain a single source of truth.

Security is another concern in traditional systems. Physical records are susceptible to damage, loss, or unauthorized access, while basic digital files may not have proper access controls. This can compromise sensitive information and affect the reliability of the system.

In addition to these limitations, manual systems do not support scalability. As the size of the organization grows and the volume of transactions increases, managing inventory manually becomes increasingly complex and unmanageable. This highlights the need for a more efficient and automated solution.

Limitations of the Existing System

- High dependency on manual data entry leading to human errors
- Lack of real-time stock updates and monitoring
- Difficulty in tracking expiry dates and batch information
- Inefficient supplier and purchase management
- Time-consuming data retrieval and report generation
- Lack of centralized data storage
- Poor data security and risk of data loss
- Not scalable for large datasets or growing organizations

CHAPTER 3

DESIGN

3.1 Entity-Relationship (ER) Design:

The Entity-Relationship (E-R) model is used to represent the logical structure of the database in the Medical Inventory Management System. It provides a clear visualization of entities, their attributes, and the relationships between them. The system is designed using a relational approach and follows normalization principles to ensure data integrity and minimize redundancy.

The main entities identified in the system include Medicine, Supplier, Inventory, Purchase Order, Purchase Details, and Sales. These entities interact with each other to represent real-world operations of a medical inventory system.

Entities and Their Attributes

1. Medicine Entity

The Medicine entity stores the fundamental details of all medicines available in the inventory.

Attributes:

- medicine_id (Primary Key)
- name
- category
- unit_price
- manufacturer
- reorder_level

Description:

This entity acts as the central component of the system. Each medicine is uniquely identified by its ID and contains essential information required for inventory management. The reorder level helps in identifying when the stock needs to be replenished.

2. Supplier Entity

The Supplier entity contains details about suppliers who provide medicines.

Attributes:

- supplier_id (Primary Key)
- name
- phone
- email
- address

Description:

This entity helps in managing supplier-related information and maintaining records of procurement sources. It is essential for tracking purchase history and ensuring reliable supply chains.

3. Inventory Entity

The Inventory entity maintains the stock details of medicines, including batch and expiry information.

Attributes:

- inventory_id (Primary Key)
- medicine_id (Foreign Key)
- batch_number
- quantity
- expiry_date
- supplier_id (Foreign Key)

Description:

This entity plays a crucial role in tracking the availability of medicines. It supports multiple batches of the same medicine, each with different expiry dates, enabling accurate stock and expiry management.

4. Purchase Order Entity

The Purchase Order entity records information about orders placed to suppliers.

Attributes:

- order_id (Primary Key)
- supplier_id (Foreign Key)
- order_date
- total_amount

Description:

This entity captures details of procurement transactions. It helps in maintaining records of purchases and tracking financial aspects of inventory acquisition.

5. Purchase Details Entity

The Purchase Details entity represents the relationship between purchase orders and medicines.

Attributes:

- order_id (Foreign Key)
- medicine_id (Foreign Key)
- quantity
- price

Primary Key: Composite Key (order_id, medicine_id)

Description:

This is a weak (associative) entity that resolves the many-to-many relationship between Purchase Order and Medicine. It stores detailed information about each item included in a purchase order.

6. Sales Entity

The Sales entity records the details of medicines sold or issued.

Attributes:

- sale_id (Primary Key)
- medicine_id (Foreign Key)
- quantity_sold
- sale_date

- total_price

Description:

This entity is used to track outgoing inventory and monitor usage patterns. It also plays a role in updating stock levels dynamically.

Relationships Between Entities

1. Medicine – Inventory Relationship

- **Type:** One-to-Many
- One medicine can have multiple inventory records (batches)

Explanation:

A single medicine may exist in multiple batches with different expiry dates. Hence, the Inventory entity stores multiple records linked to one medicine.

2. Supplier – Inventory Relationship

- Type: One-to-Many
- One supplier can supply multiple inventory batches

Explanation:

Each inventory batch is associated with a supplier, allowing the system to track the source of each stock entry.

3. Supplier – Purchase Order Relationship

- Type: One-to-Many
- One supplier can have multiple purchase orders

Explanation:

A supplier may be involved in multiple transactions over time. This relationship helps in maintaining procurement history.

4. Purchase Order – Medicine Relationship

- Type: Many-to-Many (Resolved using Purchase Details)

Explanation:

A purchase order can include multiple medicines, and a medicine can be part of multiple purchase orders. This relationship is resolved using the Purchase Details entity.

5. Medicine – Sales Relationship

- Type**:** One-to-Many
- One medicine can have multiple sales records

Fig. 3.1 ER Diagram of PMS

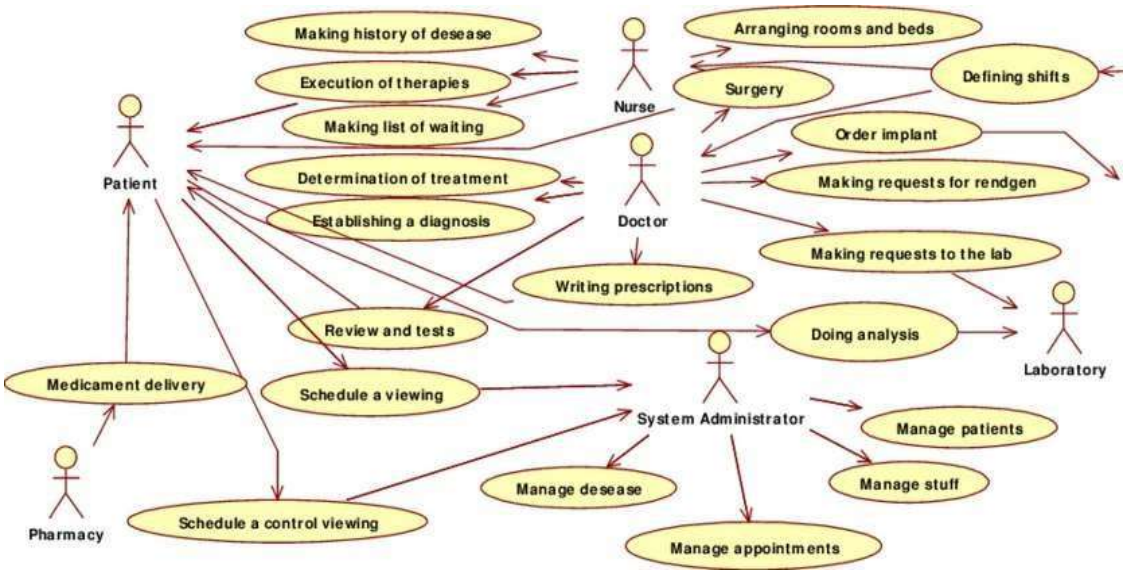

Fig. 3.2 Use Case Diagram of PMS

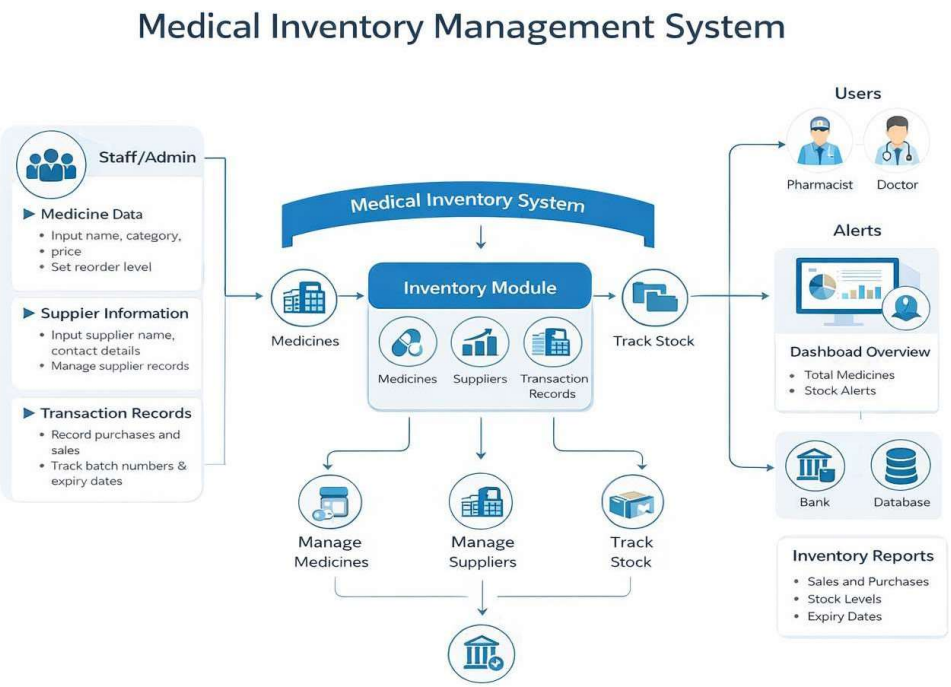

Fig. 3.3 Architecture Diagram of PMS

3.2 Front-End Design:

The front-end of the Medical Inventory Management System is designed to provide an intuitive, user-friendly, and efficient interface for interacting with the system. It serves as the presentation layer of the application, allowing users to perform various operations such as managing medicines, tracking inventory, handling suppliers, and viewing reports with ease.

The front-end is developed using standard web technologies including HTML for structuring the content, CSS for styling, and the Bootstrap framework for creating a responsive and visually appealing user interface. The design ensures compatibility across different devices and screen sizes, thereby improving accessibility and usability.

User Interface Design

The user interface is structured in a clear and organized manner to enhance user experience. The application follows a dashboard-based layout, where users can navigate between different

modules using a sidebar or navigation menu.

The main components of the interface include:

- Navigation Bar / Sidebar: Provides links to different modules such as Dashboard, Medicines, Inventory, Suppliers, Sales, and Reports.
- Dashboard Page: Displays an overview of key information such as total medicines, stock levels, and alerts for low stock or expired items.
- Forms: Used for adding and updating records such as medicines, suppliers, and transactions.
- Tables: Display structured data such as inventory lists, supplier details, and sales records in a tabular format.
- Alerts and Notifications: Highlight important information such as low stock levels or expired medicines.

The interface is designed to minimize user effort and ensure that all operations can be performed efficiently with minimal navigation.

Responsive Design

The use of Bootstrap ensures that the application is fully responsive. This means that the layout automatically adjusts based on the screen size, allowing the system to be accessed on desktop, laptops, tablets, and mobile devices without compromising usability.

Responsive design features include:

- Flexible grid system
- Adaptive layouts
- Mobile-friendly navigation menus
- Scalable UI components
- Form Handling and User Interaction

The front-end interacts with the backend using HTTP requests handled by Flask. User inputs collected through forms are sent to the server, where they are processed and stored in the database.

Key functionalities include:

- Input validation to ensure correct data entry
- Submission of forms for CRUD operations
- Dynamic updating of data displayed on the interface
- Error messages and feedback for user actions

This interaction ensures smooth communication between the user and the system.

Data Presentation

Data is presented in a structured and readable format using tables and lists. Important features include:

- Sorting and filtering of data
- Clear labeling of columns and fields
- Highlighting of critical data such as low stock and expired medicines
- Consistent formatting for better readability

This helps users quickly understand and analyse the information.

Design Considerations

While designing the front-end, the following factors were considered

- Usability: Ensuring the interface is easy to understand and navigate
- Clarity: Presenting information in a clear and organized manner
- Consistency: Maintaining uniform design across all pages
- Efficiency: Reducing the number of steps required to perform tasks
- Accessibility: Making the system usable for different types of uses

Advantages of the Front-End Design

- Provides a clean and professional user interface
- Enhances user experience through intuitive navigation
- Supports real-time interaction with the backend
- Ensures accessibility across multiple devices
- Reduces user effort and improves efficiency

CHAPTER 4

PROPOSED METHODOLOGY

4.1 Modules Description:

The system is divided into several modules, each handling a specific aspect of inventory management. These modules work together to provide a complete and integrated solution.

1. Medicine Management Module

This module is responsible for managing all information related to

medicines. Functions:

- Add new medicines
- Update existing medicine details
- Delete medicine records
- View list of medicines

Description:
It maintains essential details such as medicine name, category, price, manufacturer, and reorder level. This module acts as the core of the system, as all other modules depend on medicine data.

2. Supplier Management Module

This module handles information related to suppliers.

Functions:

- Add new suppliers
- Update supplier details
- Delete supplier records
- View supplier information

Description:
It stores supplier details such as name, contact information, and address. This helps in maintaining procurement records and managing supplier relationships efficiently.

3. Inventory Management Module

This module manages stock details of medicines.

Functions:

- Track stock levels
- Manage batch numbers
- Monitor expiry dates
- Update inventory after transactions

Description:
It keeps track of available quantities of medicines along with batch and expiry details. This ensures accurate stock monitoring and helps prevent the use of expired medicines.

4. Sales and Purchase Module

This module records all incoming and outgoing transactions.

Functions:

- Record purchase orders
- Record sales transactions
- Update stock automatically
- Maintain transaction history

Description:
It ensures that every purchase and sale is recorded in the system, and stock levels are updated accordingly. This module is crucial for maintaining inventory accuracy.

5. Dashboard and Alerts Module

This module provides an overview of system status and important

notifications. Functions:

- Display total medicines and stock
- Show low stock alerts
- Identify expired or near-expiry medicines
- Provide summary reports

Description:
It enhances decision-making by presenting key information in a simple and visual format. Alerts help users take timely actions.

Advantages of Modular Design

- Improves system organization and clarity
- Makes the system easier to maintain and update
- Allows independent development and testing of modules
- Enhances scalability for future improvements

4.2 Database Connectivity:

The system uses SQLite as the backend database, which is integrated with the application using Python. The backend framework Flask is responsible for handling communication between the user

interface and the database.

Connection Mechanism

The application establishes a connection to the SQLite database using Python's built-in database library. A connection object is created, which allows the system to execute SQL queries and interact with the database.

The process includes:

- Establishing a connection to the database file
- Creating a cursor object to execute SQL commands
- Executing queries such as SELECT, INSERT, UPDATE, and DELETE
- Committing changes to save updates
- Closing the connection after operations

Data Interaction

The backend processes user requests received from the frontend and performs corresponding database operations. Data is exchanged between the frontend and backend in JSON format.

Key operations include:

- Retrieving data from the database and displaying it on the user interface
- Inserting new records based on user input
- Updating existing records
- Deleting records when required

Data Integrity and Security

The system ensures data integrity through:

- Use of primary keys to uniquely identify records
- Use of foreign keys to maintain relationships between tables
- Validation of user inputs before storing data
- Controlled access to database operations through backend logic

Advantages of SQLite for Connectivity

- Lightweight and easy to integrate
- No need for a separate database server
- Fast data access for small to medium applications
- Suitable for standalone and web-based systems

CHAPTER 5

IMPLEMENTATION

Implementing the Medical Inventory Management System involves several structured steps, including requirement analysis, system design, development, testing, and deployment. The system is developed as a web-based application using Flask, Python, and SQLite to ensure efficient inventory management.

Identify Requirements

The first step in implementation is to analyse and understand the requirements of the system. This includes identifying the key functionalities such as managing medicines, tracking stock levels, handling supplier information, and monitoring expiry dates. The requirements are defined based on real-world challenges faced in hospitals and pharmacies.

Design the System Architecture

A suitable system architecture is designed to organize the application into different layers. The system follows a web-based architecture where the frontend interacts with the backend through HTTP requests, and the backend communicates with the database. This ensures proper separation of concerns and efficient data handling.

Set Up the Database

The database is created using SQLite, where tables are defined to store information about medicines, suppliers, inventory, purchase orders, and sales. Relationships between tables are established using primary and foreign keys to maintain data integrity. The database schema is designed based on normalization principles.

Develop Backend Functionality

The backend is implemented using Flask, where routes are created to handle different operations. These include adding new records, updating existing data, retrieving information, and deleting

records. The backend processes user requests and interacts with the database to perform the required operations.

Design the Frontend Interface

The frontend is developed using HTML, CSS, and Bootstrap to create a user-friendly and responsive interface. Forms are designed for data entry, and tables are used to display information. Navigation components are included to allow users to access different modules

Integrate Frontend and Backend

The frontend and backend are integrated using HTTP methods such as GET, POST, PUT, and DELETE. Data entered by users is sent to the backend, processed, and stored in the database. The results are then displayed back on the user interface.

Automate Inventory Processes

Automation is implemented to reduce manual effort. The system automatically updates stock levels after sales and purchase transactions. It also generates alerts for low stock and expired medicines, improving efficiency and reducing errors.

Ensure Data Integrity and Security

Measures are taken to ensure data accuracy and security. Input validation is performed to prevent incorrect data entry. Foreign key constraints are used to maintain relationships between tables. Access to database operations is controlled through backend logic.

Testing and Validation

The system is thoroughly tested to ensure that all functionalities work as expected. Testing includes verifying CRUD operations, checking database updates, and validating user inputs. Any errors identified during testing are corrected before deployment.

Deployment and Usage

After successful testing, the system is deployed and made available for use. Users can access the application through a web browser and perform inventory management operations efficiently.

Monitor and Improve Performance

The performance of the system is continuously monitored to ensure smooth operation. Feedback from users is collected to identify areas for improvement, and necessary updates are made to enhance system functionality.

5.1 Back end Implementation:

Database Setup :

```
import sqlite3

import os


DB_PATH = os.path.join(os.path.dirname(os.path.abspath(__file__)), "medical_inventory.db")


def get_connection():

    conn = sqlite3.connect(DB_PATH)

    conn.execute("PRAGMA foreign_keys = ON;")

    conn.row_factory = sqlite3.Row

    return conn
```

Table Creation:

```
def setup_db():
    conn = get_connection()
    conn.executescript("""
    CREATE TABLE IF NOT EXISTS Medicine (
        medicine_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        category TEXT NOT NULL,
        unit_price REAL NOT NULL,
        manufacturer TEXT NOT NULL,
```

```
        reorder_level INTEGER NOT NULL
    );

    CREATE TABLE IF NOT EXISTS Supplier (
        supplier_id INTEGER PRIMARY KEY AUTOINCREMENT,
        name TEXT NOT NULL,
        phone   TEXT,
        email   TEXT,
        address TEXT
    );

    CREATE TABLE IF NOT EXISTS Inventory (
        inventory_id INTEGER PRIMARY KEY AUTOINCREMENT,
        medicine_id INTEGER NOT NULL,
        batch_number TEXT NOT NULL,
        quantity INTEGER NOT NULL,
        expiry_date TEXT NOT NULL,
        supplier_id INTEGER NOT NULL,
        FOREIGN KEY (medicine_id) REFERENCES Medicine(medicine_id),
        FOREIGN KEY (supplier_id) REFERENCES Supplier(supplier_id)
    );
    """)
    conn.commit()
    conn.close()
```

Flask Application:

```
from flask import Flask, render_template, request, jsonify
from database import *

app = Flask(__name__)
setup_db()
```

API to Add Medicine:

```
@app.route("/api/medicines", methods=["POST"])
```

```
def api_add_medicine():
    data = request.get_json()
    try:
        add_medicine(
            data["name"],
            data["category"],
            float(data["unit_price"]),
            data["manufacturer"],
            int(data["reorder_level"])
        )
        return jsonify({"success": True, "message": "Medicine added successfully"})
    except Exception as e:
        return jsonify({"success": False, "message": str(e)}), 400
```

API to Get Medicines:

```
@app.route("/api/medicines", methods=["GET"])
def api_get_medicines():
    return jsonify(get_all_medicines())
```

Dashboard API:

```
@app.route("/api/dashboard")
def api_dashboard():
    return jsonify(get_dashboard_stats())
```

5.2 Front End Implementation:

HTML:

```
<!DOCTYPE html>
<html>
<head>
    <title>Medical Inventory System</title>
    <link rel="stylesheet" href="/static/style.css">
</head>
```

```
<body>
<h1>Medical Inventory Dashboard</h1>
<div id="dashboard">
   <h2>Dashboard Overview</h2>
   <div id="stats"></div>
</div>
</body>
</html>
```

JAVA SCRIPT:

```
fetch('/api/dashboard')
.then(response => response.json())
.then(data => {
   document.getElementById('stats').innerHTML =
      "Total Medicines: " + data.total_medicines;
});
```

CSS:

```
body {
   font-family: Arial, sans-serif;
   margin: 20px;
}

h1 {
   color: #2c3e50;
}
#dashboard {
   background-color: #f4f4f4;
   padding: 15px;
   border-radius: 8px;
}
```

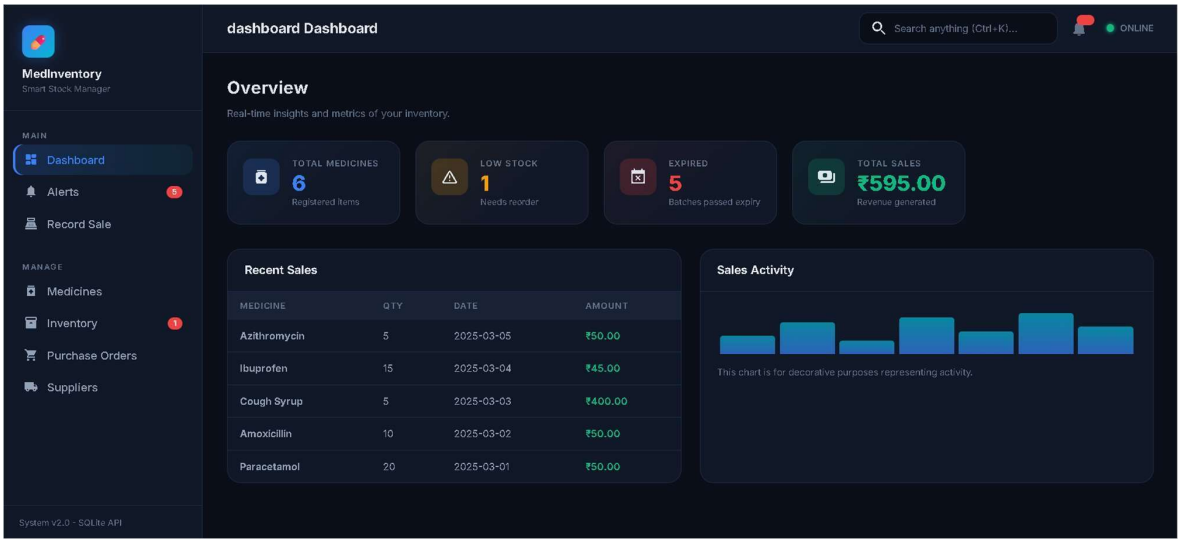

Fig. 5.1. Dashboard page

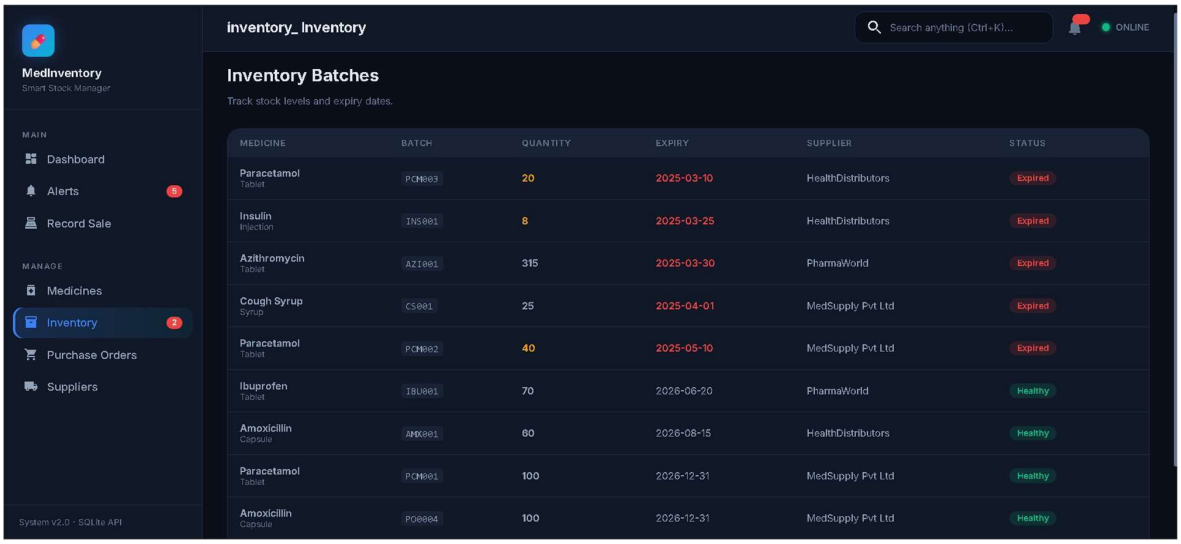

Fig. 5.2. Inventory page

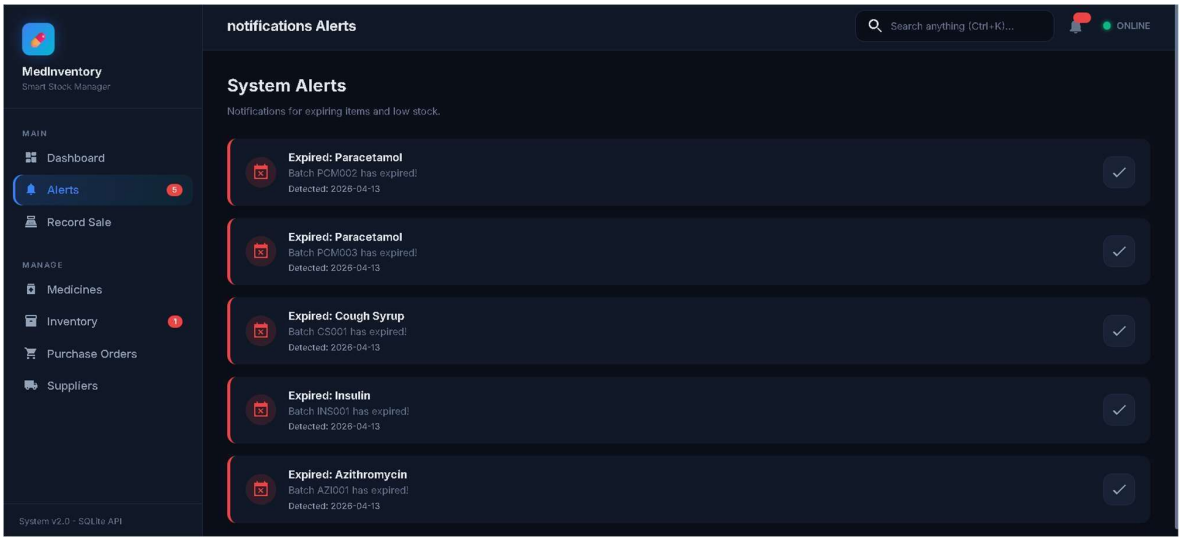

Fig. 5.3. Alerts Page

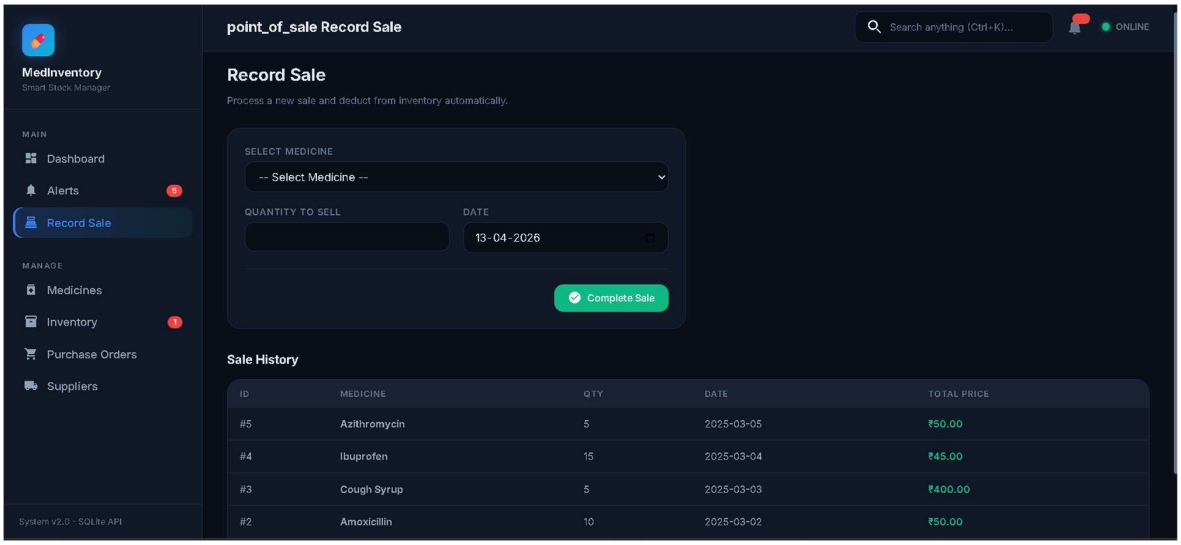

Fig. 5.4. Sale Record Page

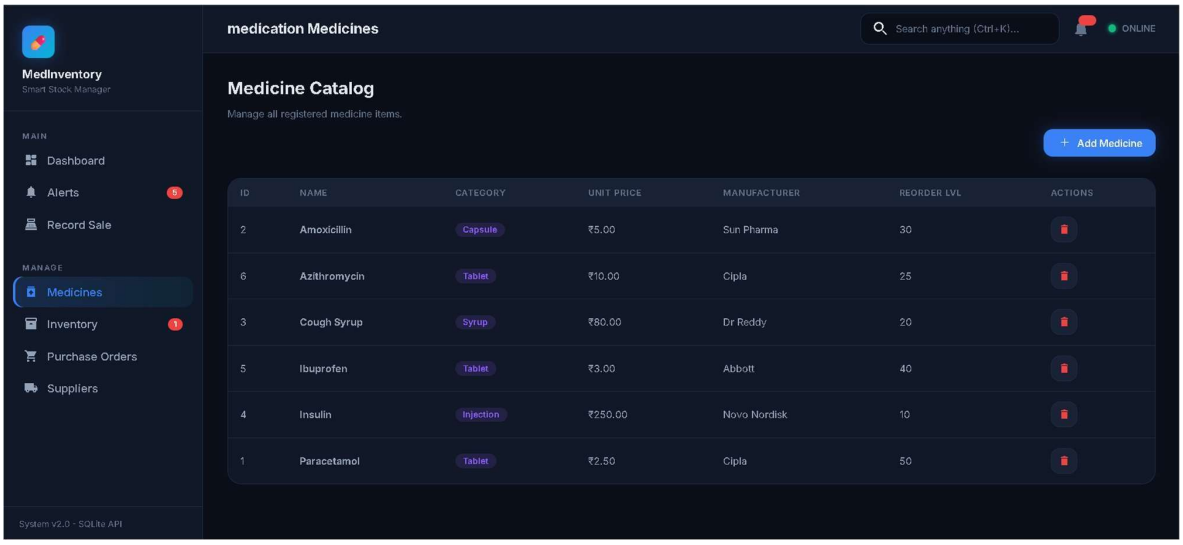

Fig. 5.5. Medicines page

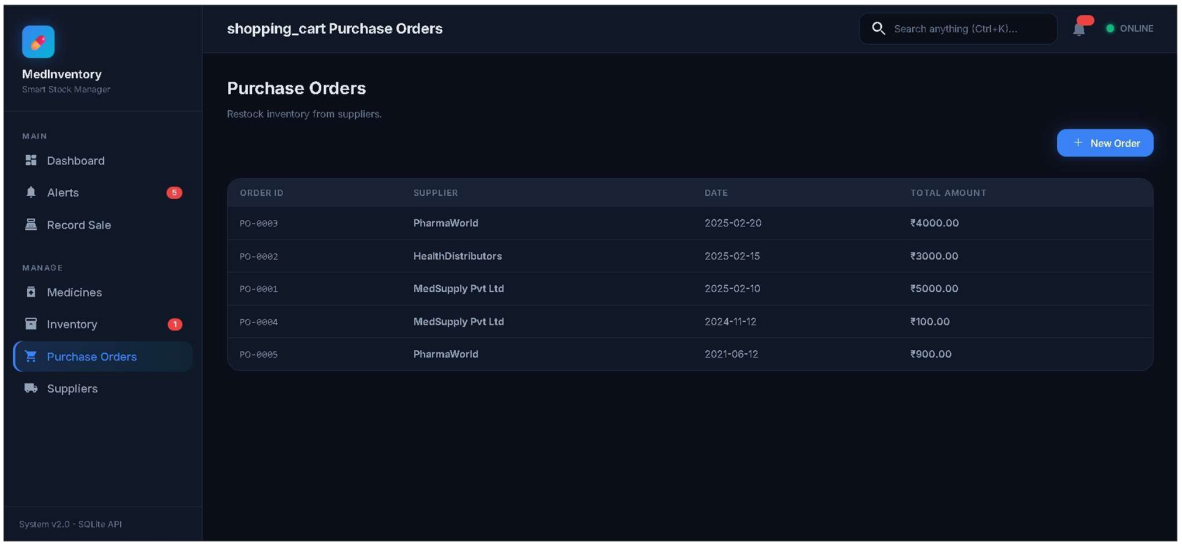

Fig. 5.6. Purchase Orders page

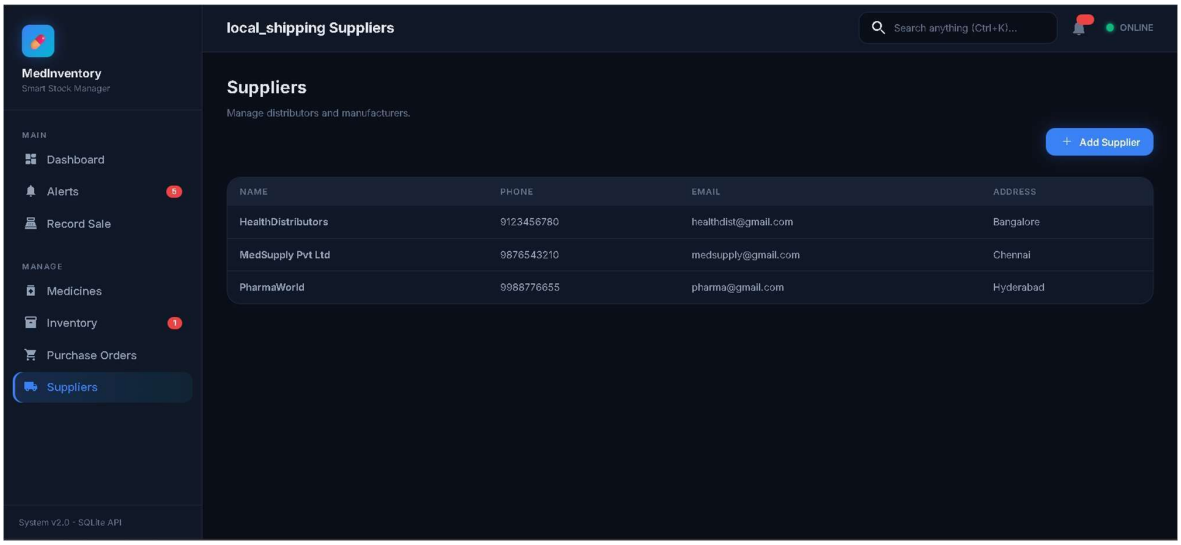

Fig. 5.7. Suppliers Page

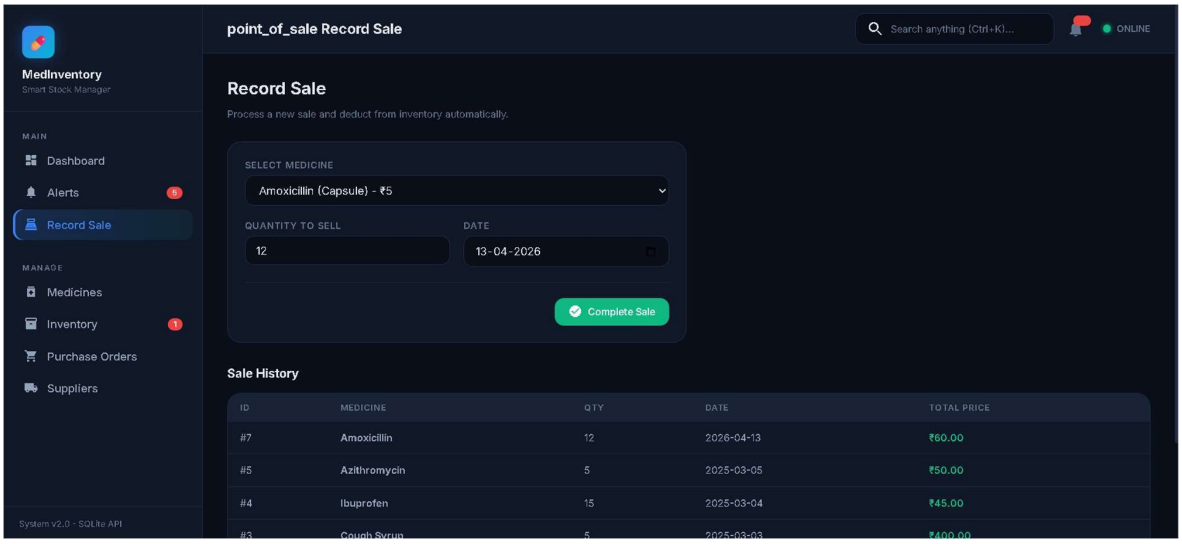

Fig. 5.8. Selling a medicine

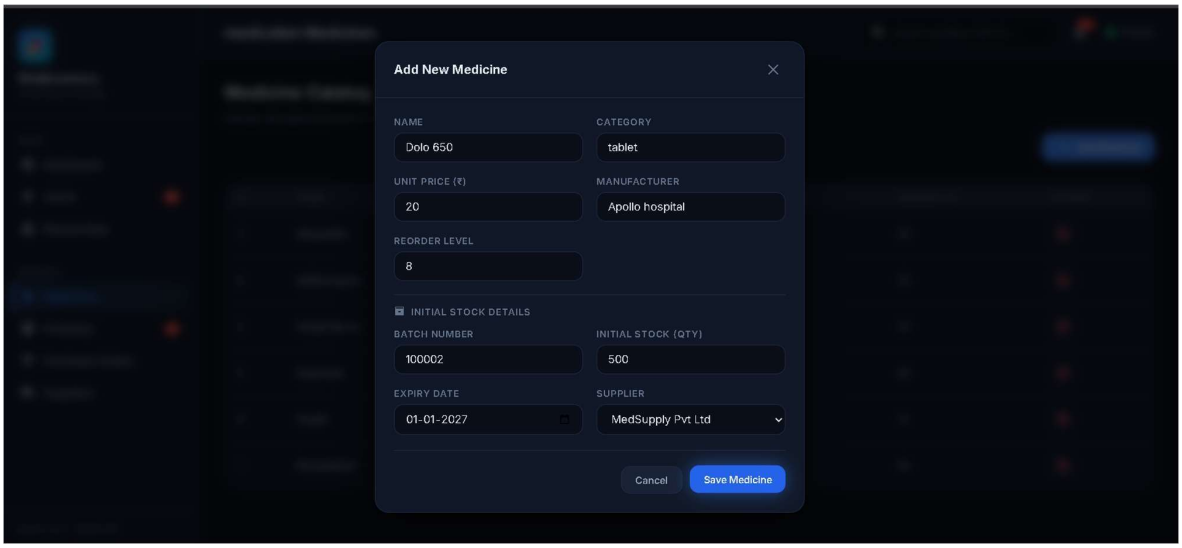

Fig. 5.9. Adding a new medicine

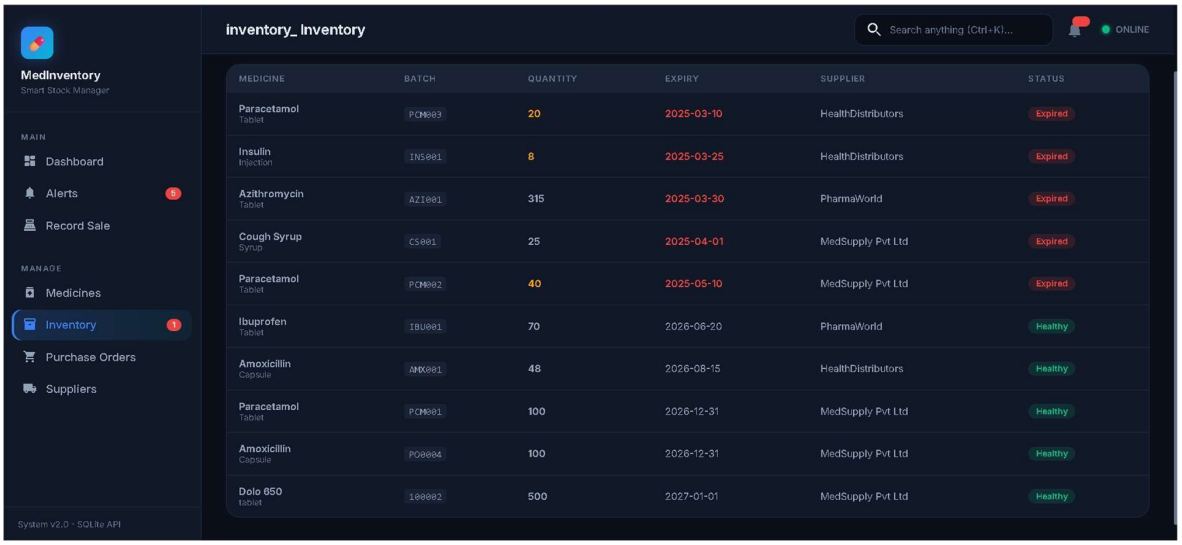

Fig. 5.. Updated Inventory

medical_inventory.db

D: > FINAL zipped > FINAL > medical_inventory.db

Filter 8 tables...

TABLES

- Alerts
- Inventory
- Medicine
- Purchase_Details
- Purchase_Order
- Sales
- sqlite_sequence
- Supplier

Rows: 10

Filter 10 rows...

	invent...	medic...	batch_nu...	quantity	expiry_date	suppli...
1	1	1	PCM001	100	2026-12-31	1
2	2	1	PCM002	40	2025-05-10	1
3	3	1	PCM003	20	2025-03-10	2
4	4	2	AMX001	48	2026-08-15	2
5	5	3	CS001	25	2025-04-01	1
6	6	4	INS001	8	2025-03-25	2
7	7	5	IBU001	70	2026-06-20	3
8	8	6	AZI001	315	2025-03-30	3
9	9	2	PO0004	100	2026-12-31	1
10	11	8	100002	0	2027-01-01	1
11						

Fig. 5.11. Inventory Database

CHAPTER 6

RESULT AND DISCUSSION

6.1 System Functionality Evaluation:

A comprehensive evaluation of the Medical Inventory Management System involves assessing its core functionalities related to inventory handling, administrative operations, system usability, and data management. The system is designed to efficiently manage medicines, suppliers, stock levels, and transactions while ensuring accuracy, reliability, and ease of use.

Core Functionalities

Medicine Information Management

The system allows users to store and manage detailed information about medicines, including name, category, price, manufacturer, and reorder level. This ensures that all medicine-related data is maintained in a structured and centralized manner.

Inventory Management

The system effectively tracks inventory by maintaining stock quantities, batch numbers, and expiry dates. It supports multiple batches for the same medicine, allowing precise monitoring of stock and preventing confusion between different batches.

Stock Monitoring and Alerts

The system automatically monitors stock levels and generates alerts when the quantity falls below the predefined reorder level. It also identifies expired or near-expiry medicines, helping users take timely action and avoid potential risks.

Supplier Management

The system maintains detailed supplier information, including contact details and transaction history. This helps in managing procurement processes efficiently and ensures accountability in supply chain operations.

Sales and Purchase Management

The system records all purchase and sales transactions, ensuring that stock levels are updated automatically. This eliminates manual calculations and reduces the chances of discrepancies in inventory data.

Reporting and Dashboard

The system provides a dashboard that displays key information such as total medicines, stock levels, and alerts. It enables users to quickly analyse the status of inventory and make informed decisions.

Administrative Functionalities

Data Management

The system allows administrators to manage all records related to medicines, suppliers, and inventory. It supports adding, updating, deleting, and retrieving data efficiently.

Stock Control

Administrators can monitor stock levels and ensure that inventory is maintained within optimal limits. This helps in avoiding both shortages and overstocking.

Transaction Handling

The system records and maintains all transactions related to purchases and sales, ensuring proper tracking and accountability.

Data Integrity

The system ensures data integrity through the use of primary keys, foreign keys, and validation mechanisms, preventing inconsistencies and duplication.

Usability and Accessibility

User-Friendly Interface

The system provides a clean and intuitive interface that allows users to navigate easily between

different modules such as medicines, inventory, suppliers, and reports.

Easy Navigation

The application includes structured menus and dashboards, enabling users to perform tasks quickly without confusion.

Responsive Design

The system is designed to work across different devices and screen sizes, ensuring accessibility and convenience for users.

Real-Time Interaction

The system allows real-time data updates and retrieval, improving efficiency and reducing delays in operations.

Security and Data Handling

Data Protection

Sensitive data such as inventory and supplier details are securely stored in the database, preventing unauthorized access.

Access Control

Database operations are handled through the backend, ensuring controlled access to

data. Data Validation

Input validation is implemented to prevent incorrect or invalid data from being stored in the system.

6.2 Performance Evaluation:

The performance of the Medical Inventory Management System is evaluated based on various parameters such as speed, accuracy, scalability, reliability, and user experience.

Speed and Efficiency

The system performs database operations such as insertion, retrieval, and updates efficiently using

SQLite. The response time for user requests is minimal, ensuring smooth operation.

Accuracy

The system accurately maintains inventory records and updates stock levels automatically after each transaction. This reduces the chances of errors and ensures reliable data management.

Scalability

The system is capable of handling moderate amounts of data efficiently. While it is suitable for small to medium-scale applications, it can be extended with more advanced databases for larger implementations.

Reliability

The system demonstrates stable performance with minimal errors during operation. Data consistency is maintained through proper database design and transaction handling.

Security

The system implements basic security measures such as controlled database access and validation checks. This ensures that sensitive data is protected from unauthorized access.

User Experience

The system provides a smooth and user-friendly experience. Users can perform tasks efficiently without requiring advanced technical knowledge.

6.3 User Feedback and Adoption:

The system was designed with a focus on usability and ease of operation. Users found the interface simple and intuitive, allowing them to perform tasks without requiring extensive technical knowledge.

The dashboard provides a clear overview of inventory status, and the structured layout of forms and tables helps users quickly navigate between different modules. The use of a web-based interface allows access through standard browsers, increasing convenience and accessibility.

Users appreciated the automation features such as real-time stock updates and alert notifications, which reduce manual effort and improve efficiency. The ability to quickly retrieve information and generate insights from the system enhances decision-making.

Areas for Improvement

Some users suggested enhancements such as:

- Adding user authentication and role-based access
- Improving visual design and interface aesthetics
- Including advanced reporting and analytics features
- Supporting integration with other systems

Overall, the system was positively received, and it demonstrates strong potential for adoption in small to medium-scale healthcare environments.

6.4 Future Directions and Enhancements:

Future directions and enhancements for the Medical Inventory Management System involve leveraging modern technologies, improving system scalability, enhancing user experience, and addressing the growing needs of healthcare institutions. By incorporating advanced features and integrations, the system can evolve into a more intelligent, secure, and efficient solution for inventory management.

Integration with AI and Machine Learning

Incorporating artificial intelligence (AI) and machine learning (ML) can significantly enhance the capabilities of the system. AI algorithms can analyse historical inventory data to identify usage patterns, predict future demand for medicines, and optimize stock levels. Machine learning models can also help in detecting anomalies such as unusual stock depletion or potential fraud, thereby improving decision-making and operational efficiency.

Mobile Accessibility

Enhancing mobile accessibility will allow users to access the system anytime and anywhere using smartphones or tablets. Developing a mobile application or optimizing the web interface for mobile devices enables pharmacists and staff to update inventory, check stock levels, and receive alerts on the go. This improves flexibility, responsiveness, and overall productivity.

Integration of Barcode and QR Code Technology

The system can be enhanced by integrating barcode scanning and QR code technology for faster and more accurate inventory management. This allows quick identification of medicines, reduces manual data entry, and minimizes errors. It also improves efficiency during stock updates, sales processing, and inventory audits.

Enhanced Data Analytics and Reporting

Improving data analytics and reporting capabilities can provide deeper insights into inventory trends, medicine usage patterns, and supplier performance. Advanced features such as interactive dashboards, graphical visualizations, and predictive analytics can help users make informed decisions, optimize stock levels, and reduce wastage.

Integration with Healthcare and Accounting Systems

Seamless integration with other systems, such as electronic health records (EHR) and accounting software can streamline operations and improve data consistency. This enables automatic synchronization of patient prescriptions, billing information, and inventory updates, reducing redundancy and enhancing overall efficiency.

Cloud Integration and Scalability

Migrating the system to cloud-based platforms can improve scalability, accessibility, and data availability. Cloud integration allows multiple users from different locations to access the system simultaneously, making it suitable for large hospitals and pharmacy chains. It also ensures better data backup and disaster recovery mechanisms.

Enhanced Security Measures

Strengthening security features is essential to protect sensitive medical and inventory data. Implementing multi-factor authentication, data encryption, and role-based access control ensures that only authorized users can access specific functionalities. Regular security updates and monitoring can further safeguard the system against cyber threats.

Automated Notification System

The system can be enhanced by implementing automated notifications through email or SMS for

events such as low stock levels, expiry alerts, and order confirmations. This ensures timely action and improves operational efficiency.

User Interface and Experience Improvements

Improving the UI/UX design can make the system more visually appealing and easier to use. Features such as customizable dashboards, better navigation, and modern design elements can enhance user satisfaction and adoption.

Future Vision

By adopting these advancements, the Medical Inventory Management System can evolve into a comprehensive, intelligent, and scalable solution capable of meeting the dynamic requirements of modern healthcare environments. Continuous improvement and integration of emerging technologies will ensure that the system remains efficient, reliable, and future-ready.

CHAPTER 7

CONCLUSION

The Medical Inventory Management System developed in this project successfully demonstrates the application of database management concepts in solving real-world healthcare challenges. The system provides an efficient and reliable solution for managing medical inventory by automating key processes such as medicine tracking, stock management, supplier handling, and transaction recording. By replacing traditional manual methods with a centralized and structured database system, the application significantly reduces human errors, improves data accuracy, and enhances operational efficiency. The use of **Flask**, **Python**, and **SQLite** ensures that the system is lightweight, easy to implement, and suitable for small to medium-scale healthcare environments. The project also highlights the importance of proper database design, normalization, and system integration in building scalable and maintainable applications. Overall, the system meets its intended objectives and provides a strong foundation for further enhancements, making it a practical and valuable tool for improving inventory management in the healthcare sector.

CHAPTER 8

REFERENCES

1. Database System Concepts, Abraham Silberschatz, Henry F. Korth, S. Sudarshan, McGraw-Hill Education.
2. Fundamentals of Database Systems, Ramez Elmasri, Shamkant B. Navathe, Pearson Education.
3. Python Documentation, Available at: https://docs.python.org/
4. Flask Documentation, Available at: https://flask.palletsprojects.com/
5. SQLite Documentation, Available at: https://www.sqlite.org/docs.html
6. Bootstrap Documentation, Available at: https://getbootstrap.com/
7. "Introduction to Database Management Systems," TutorialsPoint, Available at: https://www.tutorialspoint.com/dbms/
8. "Medical Inventory Management Systems," GeeksforGeeks, Available at: https://www.geeksforgeeks.org/
9. Design and Implementation of Medical Inventory Management System, International Journal of Computer Applications (IJCA).
10. A Study on Inventory Management System in Healthcare Sector, International Journal of Scientific Research in Computer Science.
11. Web-Based Inventory Management System Using Flask Framework, International Journal of Engineering Research & Technology (IJERT)..
12. Application of Database Management Systems in Healthcare, Springer Journal of Health Informatics.
13. Inventory Control Techniques and Their Impact on Healthcare Management, Elsevier ScienceDirect.